Network Flow I

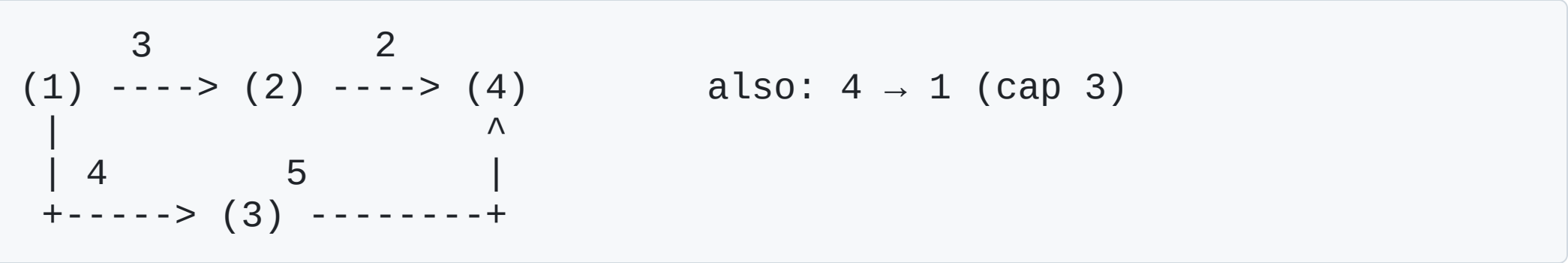
Max flow, min cut, and bipartite matching

Learning goals

- Define flow networks, feasible flows, and the **residual graph**
- Understand **augmenting paths** and why reverse edges are needed
- Implement **Edmonds-Karp** (BFS augmenting paths) and **Dinic**
- State and use **max-flow = min-cut**; recover the cut itself
- Model problems as flow: super source/sink, vertex capacities, disjoint paths
- Solve **bipartite matching** with flow or Kuhn's algorithm; know König's theorem

Motivating problem

> CSES 1694 Download Speed: a network of n ≤ 500 computers and m ≤ 1000 directed links with capacities. What is the maximum speed at which computer 1 can send data to computer n?



Answer: 2 via 1→2→4 plus 4 via 1→3→4 = **6**.

Definitions

A **flow network**: directed graph, source s, sink t, capacity $c(u,v) \geq 0$.

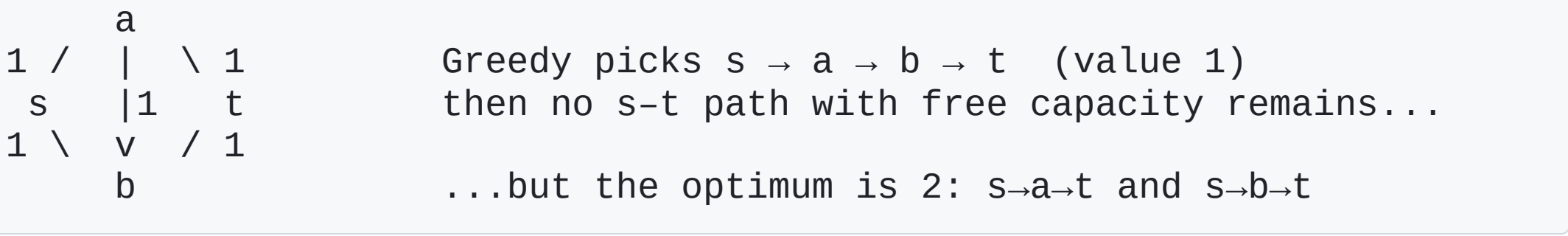
A **flow** f assigns $f(u,v)$ to each edge with:

- **Capacity:** $0 \leq f(u,v) \leq c(u,v)$
- **Conservation:** for every $v \neq s, t$: flow in = flow out

Value $|f|$ = net flow out of s = net flow into t.

Max flow problem: maximize $|f|$.

Greedy fails



We need a way to **undo** flow: route new flow "backwards" along a→b, cancelling it.

Residual graph

For each edge (u,v) with capacity c and flow f:

- **forward** residual edge u→v with capacity c − f (more can be pushed)
- **backward** residual edge v→u with capacity f (flow can be cancelled)

An **augmenting path** = an s→t path in the residual graph with positive capacities.

Push δ = min residual capacity along it: forward edges +δ, backward edges −δ. The value grows by δ.

In code: store each edge with its **twin**; pushing δ does `cap[e] -= δ; cap[e^1] += δ`.

Ford-Fulkerson method

```
while there is an augmenting path P in the residual graph:
    δ = bottleneck(P)
    augment along P by δ
```

- Integer capacities ⇒ each step adds ≥ 1 ⇒ terminates after ≤ $|f^*|$ steps: $O(E \cdot |f|)$*
- The final flow is **integral** (integrality theorem) — crucial for matching
- Arbitrary path choice can be slow (or not terminate with irrational capacities)

Edmonds-Karp

Choose the **shortest** augmenting path (BFS).

- Distances from s in the residual graph never decrease
- Each edge can be the bottleneck O(V) times ⇒ **O(VE)** augmentations
- Total **$O(V E^2)$** — independent of capacities

Fine for V, E up to a few hundred / thousand. Simple to write.

Edmonds-Karp: code

```
long long maxflowEK(int s, int t) {
    long long flow = 0;
    while (true) {
        vector<int> pe(n, -1);              // edge used to reach v
        queue<int> q; q.push(s); pe[s] = -2;
        while (!q.empty() && pe[t] == -1) {
            int v = q.front(); q.pop();
            for (int id : g[v])
                if (cap[id] > 0 && pe[to[id]] == -1) { pe[to[id]] = id; q.push(to[id]); }
        }
        if (pe[t] == -1) return flow;
        long long d = LLONG_MAX;
        for (int v = t; v != s; v = to[pe[v] ^ 1]) d = min(d, cap[pe[v]]);
        for (int v = t; v != s; v = to[pe[v] ^ 1]) { cap[pe[v]] -= d; cap[pe[v] ^ 1] += d; }
        flow += d;
    }
}
```

`to[id ^ 1]` is the tail of edge `id` (its twin points back).

Dinic's algorithm: idea

Do many augmentations per BFS:

1. **BFS** from s on the residual graph → `level[v]`
2. **Level graph**: only edges u→v with `level[v] = level[u] + 1`
3. Find a **blocking flow** in the level graph with DFS (every s→t path in it gets a saturated edge)
4. Repeat until t is unreachable

Each phase strictly increases dist(s, t) ⇒ at most V phases.

Current-arc optimization: `it[v]` remembers the first edge of v not yet known to be useless.

Dinic: code (1/2)

```
struct Dinic {
    struct Edge { int to; long long cap; };
    vector<Edge> e;
    vector<vector<int>> g;
    vector<int> level, it;
    Dinic(int n) : g(n), level(n), it(n) {}
    int addEdge(int u, int v, long long c, long long rc = 0) {
        g[u].push_back(e.size()); e.push_back({v, c});
        g[v].push_back(e.size()); e.push_back({u, rc});
        return (int)e.size() - 2;           // id of the forward edge
    }
    bool bfs(int s, int t) {
        fill(level.begin(), level.end(), -1);
        queue<int> q; q.push(s); level[s] = 0;
        while (!q.empty()) {
            int v = q.front(); q.pop();
            for (int id : g[v])
                if (e[id].cap > 0 && level[e[id].to] < 0) {
                    level[e[id].to] = level[v] + 1; q.push(e[id].to);
                }
        }
        return level[t] >= 0;
    }
```

Dinic: code (2/2)

```
    long long dfs(int v, int t, long long f) {
        if (v == t) return f;
        for (int& i = it[v]; i < (int)g[v].size(); i++) {
            int id = g[v][i], u = e[id].to;
            if (e[id].cap > 0 && level[u] == level[v] + 1) {
                long long d = dfs(u, t, min(f, e[id].cap));
                if (d > 0) { e[id].cap -= d; e[id ^ 1].cap += d; return d; }
            }
        }
        return 0;
    }
    long long maxflow(int s, int t) {
        long long flow = 0;
        while (bfs(s, t)) {
            fill(it.begin(), it.end(), 0);
            while (long long f = dfs(s, t, LLONG_MAX)) flow += f;
        }
        return flow;
    }
};
```

Dinic: complexity

Network	Bound
general	**$O(V^2 E)$**
unit capacities	$O(E \cdot \min(\sqrt{E}, V^{2/3}))$
unit network (bipartite matching)	**$O(E \sqrt{V})$**

In practice Dinic is much faster than its bound — the default choice for contests.

Cuts

An **s-t cut** (S, T): partition of vertices with s ∈ S, t ∈ T.
Capacity = Σ c(u,v) over edges u ∈ S, v ∈ T (only forward edges!).

- Any flow ≤ any cut: all flow must cross from S to T
- **Max-flow min-cut theorem:** max |f| = min cut capacity

Example: S = {1, 2}: edges 1→3 (4) + 2→4 (2) = **6** = max flow. ✓

Finding the min cut

After max flow, no augmenting path exists. Let

S = vertices reachable from s in the residual graph

- Every original edge S→T is **saturated**, every T→S edge carries 0
- So cut(S, T) = $|f|$ ⇒ it is a minimum cut

```
// after maxflow: level[v] >= 0  ⇔  v reachable (last BFS failed to reach t)
for each original edge (u, v): if (level[u] >= 0 && level[v] < 0) → in the cut
```

CSES 1695 Police Chase: undirected, unit capacities → print the cut edges.

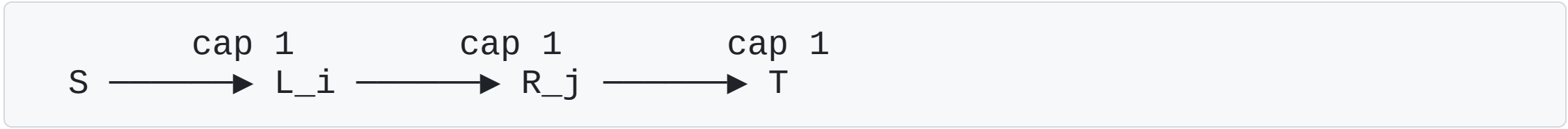
Modeling toolbox

Situation	Trick
many sources / sinks	super source $S \to s_i$, $t_i \to$ super sink T (cap ∞ or supply)
undirected edge cap c	one edge pair with cap c **both** ways
vertex capacity c(v)	split v into v_in → v_out with cap c(v)
edge-disjoint s-t paths	all caps 1; answer = max flow
vertex-disjoint paths	split vertices with cap 1
"cut cheapest set"	min cut — identify what "separating" means

Recover actual paths by walking edges with flow (CSES 1711 Distinct Routes).

Bipartite matching as flow

Left set L, right set R, allowed pairs E.



- Integral max flow ⇒ each L_i, R_j used at most once ⇒ a **matching**
- Max flow = **maximum matching** size
- Dinic runs in O(E √V) here (Hopcroft-Karp bound)

Matched pairs: L→R edges with flow 1 (residual cap 0).

Kuhn's algorithm (simpler matching)

```
vector<vector<int>> adj;  // adj[l] = right vertices
vector<int> matchR;       // matchR[r] = left partner or -1
vector<char> used;
bool tryKuhn(int l) {
    for (int r : adj[l]) if (!used[r]) {
        used[r] = 1;
        if (matchR[r] == -1 || tryKuhn(matchR[r])) { matchR[r] = l; return true; }
    }
    return false;
}
// for each l: fill(used, 0); if (tryKuhn(l)) ++matching;
```

Augmenting path search from each left vertex: **O(V · E)**. Usually fast enough for V ≤ 10^3–10^4.

König's theorem

In a **bipartite** graph:

- min **vertex cover** = max matching
- max **independent set** = $|V|$ − max matching
- min **edge cover** = $|V|$ − max matching (no isolated vertices)

Cover construction: from the min cut — left vertices **not** reachable + right vertices reachable from S in the residual graph.

Classic: "place rooks / guards so that no two attack" → independent set in a bipartite graph.

Worked modeling example

> n workers, m jobs. Worker i can do a listed set of jobs and at most k_i jobs in total; job j needs r_j workers. Workers belong to departments; department d may take at most D_d jobs overall. Maximize assignments.

```
S --D_d--> dept d --k_i--> worker i --1--> job j --r_j--> T
```

- One unit of flow = **one (worker, job) assignment**
- Each capacity limits exactly one quantity: check every constraint maps to one edge
- Worker → job capacity 1: a worker cannot take the same job twice

Complexity summary

Algorithm	Time	Use
Ford-Fulkerson (DFS)	$O(E \cdot \lvert f \rvert)$	small flow values
Edmonds-Karp	$O(V E^2)$	small graphs, easy code
Dinic	$O(V^2 E)$; $O(E\sqrt{V})$ unit	**default**
Kuhn	$O(V E)$	bipartite matching, short code
Hopcroft-Karp	$O(E \sqrt{V})$	big matchings (Nationals 5)

Common pitfalls

- Forgetting the **reverse edge** (or giving it capacity c for a directed edge)
- `int` capacities overflowing — flow sums can exceed 2^{31}
- Edge ids: twins must be **adjacent** (id, id^1)
- Undirected edge as two separate pairs is OK but doubles memory; one pair with cap c both ways is enough
- Reading the min cut from the wrong BFS (must be **after** the final, failing BFS)
- Kuhn: forgetting to reset `used` for each new left vertex
- Modeling: capacity on the wrong object (vertex vs edge)

Practice problems

- **CSES 1694** Download Speed — max flow
- **CSES 1695** Police Chase — min cut edges
- **CSES 1696** School Dance — bipartite matching
- **CSES 1711** Distinct Routes — edge-disjoint paths + reconstruction
- **Kattis maxflow**, **Kattis mincut** — template checks
- **CF 653D** Delivery Bears — binary search on the answer + flow
- **CF 498C** Array and Operations — matching on prime factors
- **SPOJ FASTFLOW** — needs a fast algorithm

Summary

- Residual graph + augmenting paths; reverse edges let flow be undone
- Edmonds-Karp $O(VE^2)$; **Dinic** $O(V^2E)$, $O(E\sqrt{V})$ on unit networks
- **Max flow = min cut**; the cut = vertices reachable from s afterwards
- Modeling: super nodes, vertex splitting, unit capacities for disjointness
- Bipartite matching via flow or Kuhn; König: cover = matching

Next week: game theory — win/lose states, Nim, Sprague-Grundy.